Parcial 2 – Fundamentos de Inteligencia Artificial

Puntos 2 y 3: Ascenso de colinas y Temple simulado

Jacobo Mondragon

14 de octubre de 2026

1. Punto 2. Ascenso de colinas

Se maximiza f sobre $x \in \{0, \ldots, 8\}$. Vecinos: $x \pm 1$ dentro del dominio. Solo se acepta mejora estricta.

x	0	1	2	3	4	5	6	7	8
$f(x)$	1	4	7	5	3	6	9	8	5

1.1. A. Análisis

1–2. Ejecución manual.

x_0	Trayectoria	Criterio de parada
0	$0 \to 1 \to 2$	en 2 los vecinos valen 4 y 5, ambos < 7
4	$4 \to 5 \to 6$	en 6 los vecinos valen 6 y 8, ambos < 9

3. Máximos.

- Locales: $x = 2$ ($f = 7$) y $x = 6$ ($f = 9$).
- Global: $x = 6$ ($f = 9$).

4. Dependencia del estado inicial. El algoritmo es determinista y solo evalúa vecinos inmediatos. Cada máximo local tiene una cuenca de atracción: $\{0, 1, 2, 3\} \to 2$ y $\{4, 5, 6, 7, 8\} \to 6$. Cambiar x_0 de cuenca cambia el resultado.

5. Estrategia. Reinicios aleatorios (*random restart*): k ejecuciones desde estados iniciales aleatorios, conservando el mejor resultado. Costo: k veces más evaluaciones de f, sin garantía de hallar el óptimo global.

1.2. B. Implementación

```
VALORES = [1, 4, 7, 5, 3, 6, 9, 8, 5]


def ascenso_colinas(valores, inicio):
    """Devuelve el estado final y la trayectoria."""
    x = inicio
    trayectoria = [x]
    while True:
        vecinos = [v for v in (x - 1, x + 1) if 0 <= v < len(valores)]
        mejor = max(vecinos, key=lambda v: valores[v])
```

```
        if valores[mejor] > valores[x]:  # solo mejora estricta
            x = mejor
            trayectoria.append(x)
        else:
            return x, trayectoria


if __name__ == "__main__":
    print(f"{'Inicio':<7}{'Trayectoria':<16}{'Final':<7}{'f(final)':<8}")
    global_max = max(VALORES)
    aciertos = 0
    for inicio in range(len(VALORES)):
        final, tray = ascenso_colinas(VALORES, inicio)
        print(f"{inicio:<7}{'->'.join(map(str, tray)):<16}{final:<7}{VALORES[
            final]:<8}")
        aciertos += VALORES[final] == global_max
    n = len(VALORES)
    print(f"\nLlegan al máximo global: {aciertos}/{n} = {aciertos / n:.1%}")
```

Resultados ($x_0 = 0, \ldots, 8$)**.**

Inicio	Trayectoria	Final	f(final)
0	$0 \to 1 \to 2$	2	7
1	$1 \to 2$	2	7
2	2	2	7
3	$3 \to 2$	2	7
4	$4 \to 5 \to 6$	6	9
5	$5 \to 6$	6	9
6	6	6	9
7	$7 \to 6$	6	9
8	$8 \to 7 \to 6$	6	9

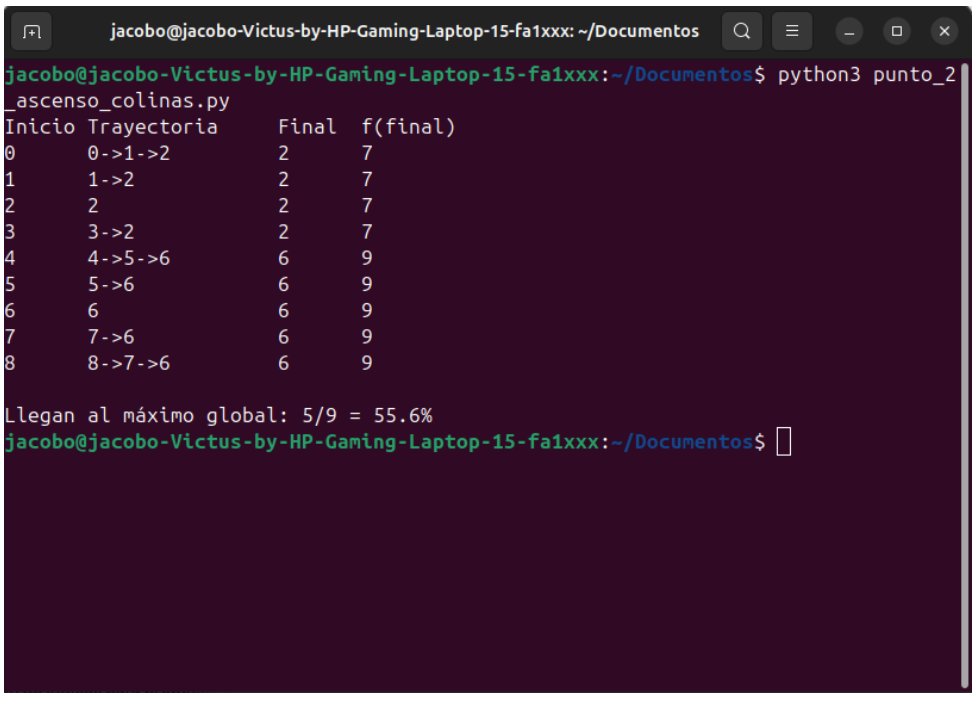


Figura 1: Ejecución de `punto_2_ascenso_colinas.py`.

2. Punto 3. Temple simulado

Se minimiza $E(x) = x^2$ con $x \in \{-5, \ldots, 5\}$, con $\Delta E = E(x_{\text{nuevo}}) - E(x_{\text{actual}})$ y

$$P(\text{aceptar}) = \begin{cases} 1, & \Delta E \leq 0, \\ e^{-\Delta E/T}, & \Delta E > 0. \end{cases}$$

Una solución peor se acepta si $u < P(\text{aceptar})$, con $u \sim U[0, 1)$.

2.1. A. Análisis

1. Casos.

Caso	Transición	T	u	ΔE	P	Decisión
A	$3 \to 2$	5	0,70	-5	1	Acepta
B	$2 \to 3$	5	0,30	$+5$	$e^{-1} \approx 0{,}3679$	Acepta ($u < P$)
C	$2 \to 3$	1	0,30	$+5$	$e^{-5} \approx 0{,}0067$	Rechaza ($u > P$)

2. Efecto de la temperatura. T alta: $P \to 1$, se aceptan casi todos los movimientos peores (exploración). T baja: $P \to 0$, solo se aceptan mejoras (explotación). En B y C, ΔE y u son iguales y la decisión cambia solo por T.

3. Enfriamiento demasiado rápido. T se acerca a 0 antes de explorar lo suficiente, el algoritmo deja de aceptar empeoramientos y se comporta como ascenso de colinas, con riesgo de quedar en un óptimo local.

4. Comparación con ascenso de colinas. El ascenso de colinas es determinista y solo acepta mejoras. El temple simulado acepta empeoramientos con probabilidad $e^{-\Delta E/T}$, decreciente con el enfriamiento, y por eso puede escapar de óptimos locales.

2.2. B. Implementación

```
import math
import random


def energia(x):
    return x ** 2


def casos_manuales():
    print("Casos␣A,␣B,␣C")
    casos = [("A", 3, 2, 5, 0.70), ("B", 2, 3, 5, 0.30), ("C", 2, 3, 1, 0.30)]
    for nombre, act, cand, T, u in casos:
        dE = energia(cand) - energia(act)
        p = 1.0 if dE <= 0 else math.exp(-dE / T)
        decision = "ACEPTA" if (dE <= 0 or u < p) else "RECHAZA"
        print(f"␣␣{nombre}:␣dE={dE:+d}␣␣P={p:.4f}␣␣u={u:.2f}␣␣->␣{decision}")
    print()


def temple_simulado(x0=5, T0=10.0, alfa=0.90, iteraciones=100, semilla=42):
    random.seed(semilla)
    x, T = x0, T0
    mejor = x
    hist_T, hist_E = [], []
```

```
    for _ in range(iteraciones):
        vecinos = [v for v in (x - 1, x + 1) if -5 <= v <= 5]
        cand = random.choice(vecinos)
        dE = energia(cand) - energia(x)
        if dE <= 0 or random.random() < math.exp(-dE / T):
            x = cand
        if energia(x) < energia(mejor):
            mejor = x
        hist_T.append(T)
        hist_E.append(energia(x))
        T *= alfa
    return mejor, hist_T, hist_E


if __name__ == "__main__":
    casos_manuales()
    mejor, hist_T, hist_E = temple_simulado()
    print(f"Mejor␣solución:␣x␣=␣{mejor}")
    print(f"Energía␣mínima:␣E␣=␣{energia(mejor)}")
    print("Historial␣de␣temperatura:", [round(t, 4) for t in hist_T])
    print("Historial␣de␣energía:", hist_E)
```

Parámetros. $x_0 = 5$, $T_0 = 10$, $T_{k+1} = 0{,}90\,T_k$, candidato aleatorio entre $x \pm 1$ válidos, 100 iteraciones, semilla 42.

Resultados (semilla 42).

- Mejor solución: $x = 0$.
- Energía mínima: $E = 0$.
- Temperatura: $T_k = 10 \cdot 0{,}9^{\,k-1}$, con $T_{100} \approx 0{,}0003$.
- Energía del estado actual: llega a 0 en la iteración 9 y permanece en 0 desde la 27.

El historial completo de temperatura y energía se muestra en la Figura 2.

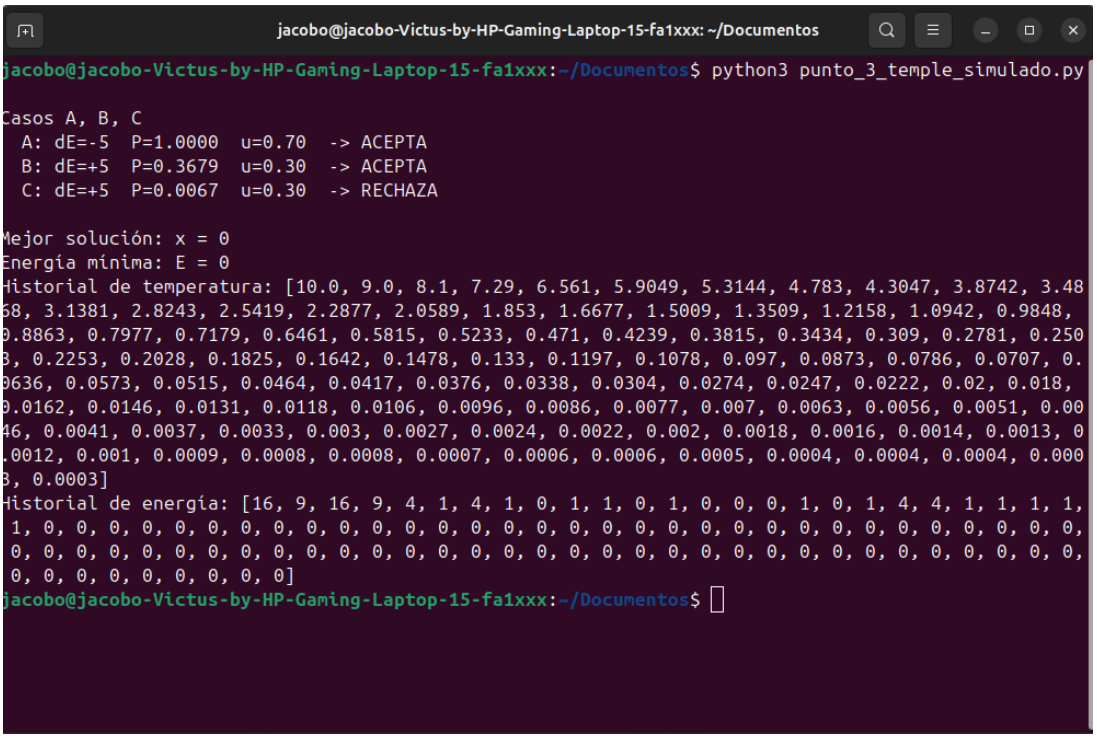


Figura 2: Ejecución de `punto_3_temple_simulado.py`.